

Contents

Multi-Perspective Advisory: Core Architecture & Session Management (Phases 0-2 Focus)	1
1. Architecture Perspective (Lead: Core Architect Planner)	1
2. Security Perspective (Lead: Security Planner/Auditor)	1
3. Performance & Scalability Perspective (Lead: Performance Planner/Auditor)	2
4. Concurrency & Error Handling Perspective (Lead: Concurrency Planner/Auditor)	2
5. Maintainability & Idiomatic Rust Perspective (Lead: Maintainability Auditor)	2
6. Integration with Grok Build & Ecosystem Perspective (Lead: Integration Auditor)	2
Overall Recommendations for Implementation & Revise Loops	3

Multi-Perspective Advisory: Core Architecture & Session Management (Phases 0-2 Focus)

For: Grok Build Tauri Control Panel Backend

Perspectives: Architecture, Security, Performance, Concurrency, Maintainability, Integration with Grok Build

Date: July 10, 2026

Status: Post-Planning / Pre-Implementation Advisory (to be updated in revise loops)

1. Architecture Perspective (Lead: Core Architect Planner)

Strengths of Proposed Design: - Clean separation: `grok_control_core` (orchestration), `grok_acp` (protocol), `grok_cli_wrapper` (CLI), `grok_events` (bus), `grok_config`. - Workspace structure allows independent compilation/testing. - `SessionRegistry` as central state with `Arc` for safe concurrent access. - `AgentHandle` abstraction unifies ACP (preferred long-lived) and Headless modes. - Event-driven: broadcast channels for real-time tool/plan/status updates to Tauri frontend.

Recommendations: - Use `DashMap` or `tokio::sync::RwLock` with care for high concurrency (report mentions up to 8 subagents + multi-session). - Add persistence layer early (SQLite via `sqlx` or `rusqlite`) for crash recovery (Phase 4). - Define clear traits: `AgentHandleTrait`, `WorktreeManager`, `PermissionController` for extensibility. - Mermaid diagram in `code_sketches` for data flow.

Risks: Over-abstraction leading to complexity. Mitigate with focused impls first.

Score: 9/10 - Solid foundation matching report's "first-class backend process" vision.

2. Security Perspective (Lead: Security Planner/Auditor)

Strengths: - Sandbox profiles (`workspace/read-only/strict`) from report directly mapped to `SpawnOptions`. - Permission rules (`allow/deny` per-tool like `Bash(git *)`, `Write(src/**)`) enforced at spawn and per-session. - "Trust this repo" flag to reduce chatty prompts. - Process isolation: `kill_on_drop`, clean env (strip dangerous vars), capture stderr. - Plan mode as "soft gate" for complex tasks (mandatory approval before writes).

Recommendations & Mitigations: - **Critical:** Never default to `-always-approve` / `-yolo`. Force plan mode for non-trivial (auditor flag). - Validate all user inputs to spawn (`cwd`, rules) to prevent command injection in CLI wrappers. - For worktrees: Ensure git operations are sandboxed; use `-ref` to control base. - OS-level sandboxing on top (Phase 4): macOS seatbelt, Linux `seccomp/namespace`s, Windows Job Objects. Conditional compile. - Audit all file ops in ACP Client (fs capabilities limited). - Secrets: Never log `XAI_API_KEY`; use env only.

Risks: Agent full shell access by design (report). Mitigate with strict defaults + user "trust" flow. **Score:** 8.5/10 - Strong model, needs rigorous enforcement in revise loops.

3. Performance & Scalability Perspective (Lead: Performance Planner/Auditor)

Strengths: - ACP preferred over repeated headless spawns (lower overhead for interactive). - tokio async throughout: non-blocking process I/O, event loops. - Worktree isolation prevents file conflicts in parallel subagents (up to ~8). - Event bus with bounded channels to prevent memory bloat.

Recommendations: - **Hot Path:** SessionRegistry lookups - use `DashMap<SessionId, Arc>` for lock-free reads where possible. - Limit concurrent sessions (config `max_concurrent_sessions`: 10 default) to avoid resource exhaustion (report risk). - Streaming JSON in headless: Parse NDJSON efficiently (`serde` + `lines`). - For multi-agent: Spawn subagents in background tasks; monitor with `/tasks` equivalent. - Benchmarks: Simulate 20 sessions + 8 subagents; profile with `tokio-console` or `flamegraph`. - Memory: Cross-session memory (report) via efficient injection; avoid duplicating context.

Risks: Lock contention or too many processes. Mitigate with pooling or limits. **Score:** 9/10 - Excellent async foundation; watch for real-world scaling.

4. Concurrency & Error Handling Perspective (Lead: Concurrency Planner/Auditor)

Strengths: - `RwLock` for registry, broadcast for events (fan-out safe). - Proper Child management with `kill_on_drop`. - Graceful signals: `SIGINT` -> cancel, `SIGTERM` -> hard kill after timeout. - ACP cancel support.

Recommendations: - **Critical for Revise:** Ensure all handles are `Send + Sync`. Use `Arc` everywhere for sharing across tasks. - Error types: Custom enum `AcpError`, `CliError`, `SessionError` with `thiserror`. Propagate with `?`. - Timeouts: Every spawn/prompt has configurable timeout (`tokio::time::timeout`). - Deadlock prevention: Avoid nested locks; use `try_read` or structured concurrency (`tokio::select!`). - For worktrees: Git ops in dedicated tasks; handle concurrent git on same repo carefully (though isolated). - Recovery: On crash, restore from persistence + re-spawn ACP sessions if possible.

Risks: Orphan processes or stuck sessions. Mitigate with supervision tree (like `actix` or custom). **Score:** 8/10 - Good base; needs thorough testing in audit loops.

5. Maintainability & Idiomatic Rust Perspective (Lead: Maintainability Auditor)

Strengths: - Modular crates: Easy to test in isolation (e.g., mock `AcpClient`). - `Serde` for config/JSON: Type-safe. - Extensive docs/comments in sketches.

Recommendations: - Follow Rust 2021 edition, `clippy::all`, `deny(warnings)` in CI. - Use `thiserror`/anyhow consistently. - Comprehensive tests: Unit for registry, integration with mock processes. - Documentation: Every public fn has examples. Generate `rustdoc`. - Version pinning: Exact versions in `Cargo.toml` (report mentions beta flux). - Refactoring: Keep `AgentHandle` focused; extract `PermissionEngine` if grows.

Risks: Boilerplate in JSON-RPC. Mitigate with macros or typed builders. **Score:** 9.5/10 - Highly maintainable design.

6. Integration with Grok Build & Ecosystem Perspective (Lead: Integration Auditor)

Strengths: - Direct use of grok agent stdio (ACP) - highest fidelity as per report. - Full CLI subcommand support via typed `GrokCli` wrapper (`inspect`, `worktree`, `mcp`, `plugin`, `sessions`, etc.). - Config TOML fidelity: Read/write `~/.grok/config.toml` + project `.grok/`. - Ecosystem: `AGENTS.md`, `skills`, `hooks`, `MCP`, `Imagine`, `memory` all mappable. - Self-hosting potential: Control panel can drive Grok Build to help build itself!

Recommendations: - Pin Grok Build version in config; auto-update opt-in. - Expose raw JSON escape hatch for future ACP changes (beta risk). - For MCP: Full CRUD mirroring grok mcp subcommands + live toggle. - Worktrees: Leverage native git + `~/.grok/worktrees` paths exactly. - Headless fallback: Perfect for scheduler/routines.

Risks: Grok Build beta changes (flags, ACP methods). Mitigate: Version pin, adapters, changelog watcher in Orchestrator. **Score:** 9/10 - Perfect alignment with report.

Overall Recommendations for Implementation & Revise Loops

- **Priority in Phase 1:** Get ACP + single session rock-solid (core of everything).
- **In Every Revise Wave:** Re-run all auditors; fix Critical/High first. Aim for 0 issues after 2 loops max.
- **Testing Strategy:** Use mock processes for ACP/CLI in tests. Simulate multi-session with `tokio::task::spawn`.
- **Next:** Expand sketches with full Cargo.toml, Tauri invoke handlers, worktree manager, permission engine.
- **Confidence in Design:** 90% - Matches report exactly, leverages Grok strengths, production-ready skeleton.

Auditor Consensus: Proceed to Implementation Wave. Design is robust; minor refinements in revise.